

# Module 9 Week B — Applied Lab: Entity Linking on a Property Graph

**Due:** Tuesday end of day. Resubmissions are accepted through Saturday of the assignment week.

You will build an entity-linking module that takes spaCy NER spans on recipe-blog text, queries a pre-loaded ~200-node recipe property graph in Neo4j, and returns a `LinkResult` per span — either a predicted `(node_id, type_label)` pair or `NIL`. The linker reports precision / recall / F1 against a gold annotation. The function-call API (`link(driver, doc_id, text, ner_spans) -> list[LinkResult]`) is the surface the Wednesday Week B integration imports without modification. The autograder gates on test-split P/R/F1 thresholds. A reference linker meets the thresholds with margin; your linker should too if you implement candidate generation, the NER—KG type filter, the `([:SUBCLASS_OF],)` hierarchical traversal, the co-occurrence signal, and the NIL semantics cleanly.

## Setup

If you haven't already installed the Module 9 packages, go back to the [Setup: install before the concept session](#) section of the reading and complete the install before proceeding.

*Windows users:* Commands below assume `Bash/Zsh` (`macOS/Linux/WSL2/Git Bash`). On `PowerShell` or `CMD`, activate your env with `./envScripts/Activate.ps1` (`PowerShell`) or `./envScripts/activate.bat` (`CMD`), and confirm Docker Desktop is running with the `WSL2` backend before `docker compose up -d`. Full cross-platform command tables are in the *Pre-Work* backend plan.

*Submission flow* — all Module 9 Week B assignments. Week B uses a fork-and-submit flow (drill, lab, integration, both stretches) instead of Git-hub Classroom. Steps 1–3 below are the new part; steps 4+ are the same workflow you used in the drill.

- Fork the template repo to your GitHub account.** Open [github.com/LevelUp-Applied-Aim9-19b](#) and click **Fork** (top right) — set the owner to your personal GitHub account — click **Create fork**. Leave "Copy the main branch only" checked.
- Enable Actions on your fork.** On your forked repo, click the **Actions** tab. GitHub disables workflows on forks by default — click **I understand my workflows, go ahead and enable them**. (One-time per fork. The autograder cannot run on your PR until this is done.)
- Clone your fork (not the upstream repo):**

```
git clone https://github.com/<your-github-username>/a9-19b.git
cd m9-19b
```
- Create your working branch:**

```
git checkout -b lab-9b-entity-linking
```
- Bring up Neo4j:**

```
docker compose up -d
```

This starts a Neo4j 5 Community container on `localhost:7687` (Bolt) and `localhost:7474` (HTTP browser). Memory caps are pre-configured for a 16 GB laptop (1G heap, 512M page cache).
- Install dependencies** (cached from the drill — should be fast):

```
pip install -r requirements.txt
```
- Load the recipe fixture:**

```
python load_fixture.py
```

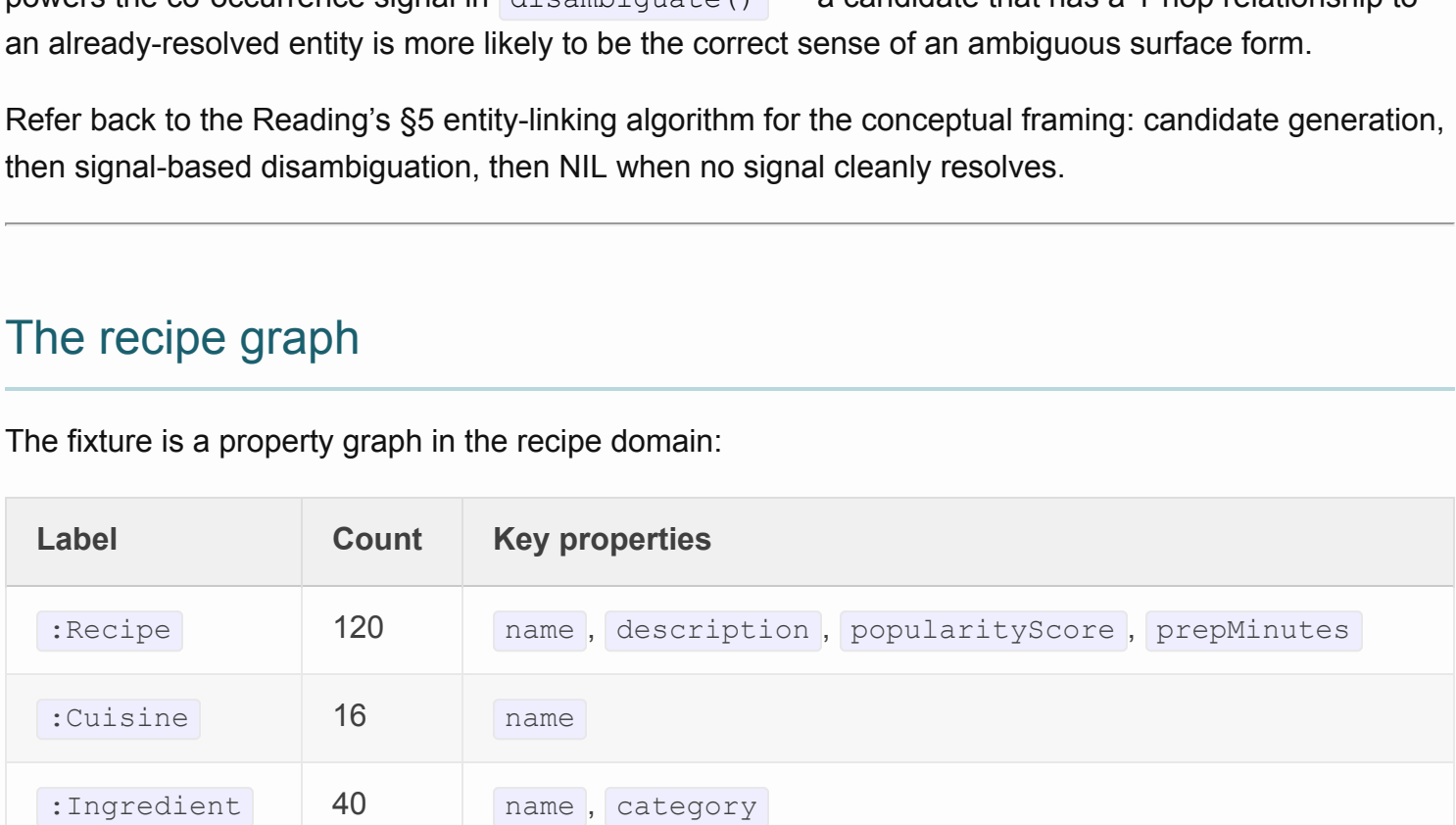
This ingests `data/recipes_kg_cypher` — the ~200-node recipe graph — and runs the acceptance Cypher (node count, relationship count, `:Entity` uniqueness). If any of those assertions fail, the script exits non-zero and the autograder will too. A clean run prints the `node/relationship` counts and the message `Fixture loaded.`
- Sanity-check the baseline:**

```
python run_dev_eval.py
```

On the unmodified starter this raises `NotImplementedError` from `candidates()` — that is expected. After you implement the first two tasks, re-run it and you will see a real P/R/F1 triple on the dev split.

## What you'll build

The data flow for one document:



The orchestrator iterates NER spans in **document order** so that `doc_resolved` (the list of already-resolved `LinkResult`s; records for this document) grows monotonically. That growing list is what powers the co-occurrence signal in `disambiguate` — a candidate that has a 1-hop relationship to an already-resolved entity is more likely to be the correct sense of an ambiguous surface form.

Refer back to the Reading's §5 entity-linking algorithm for the conceptual framing: candidate generation, then signal-based disambiguation, then NIL when no signal cleanly resolves.

## The recipe graph

The fixture is a property graph in the recipe domain:

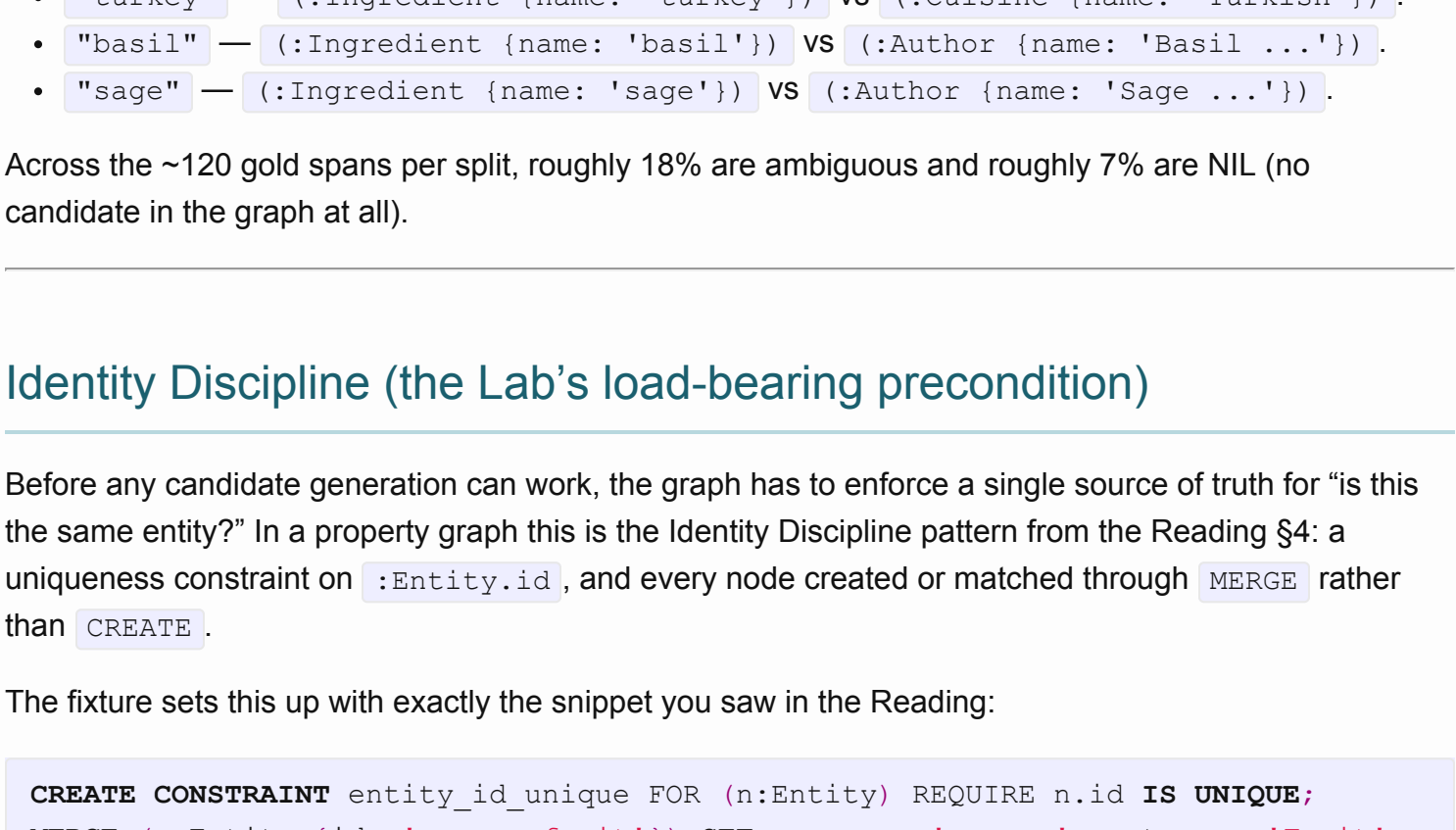
| Label                    | Count | Key properties                                                                                         |
|--------------------------|-------|--------------------------------------------------------------------------------------------------------|
| <code>:Recipe</code>     | 120   | <code>name</code> , <code>description</code> , <code>popularityScore</code> , <code>prepMinutes</code> |
| <code>:Cuisine</code>    | 16    | <code>name</code>                                                                                      |
| <code>:Ingredient</code> | 40    | <code>name</code> , <code>category</code>                                                              |
| <code>:Author</code>     | 12    | <code>name</code> , <code>country</code>                                                               |
| <code>:Technique</code>  | 12    | <code>name</code>                                                                                      |

Every domain node also carries the universal `:Entity` label and a canonical `id` of the form `<label>:<id>` (e.g. `:recipe42`, `:ingredientorange`, `:cuisine:sichuan`). The Identity Discipline section below explains why this matters.

Relationships:

- `(:Recipe)-[:USES_INGREDIENT]->(:Ingredient)`
- `(:Recipe)-[:OF_CUISINE]->(:Cuisine)`
- `(:Recipe)-[:BY_AUTHOR]->(:Author)`
- `(:Recipe)-[:REQUIRES_TECHNIQUE]->(:Technique)`
- `(:Cuisine)-[:SUBCLASS_OF]->(:Cuisine)` — the load-bearing 3–4-level hierarchy
- `(:Ingredient)-[:SUBCLASS_OF]->(:Ingredient)` — secondary hierarchy

The cuisine hierarchy is what makes the disambiguator interesting:



**Ambiguous surface forms.** The fixture deliberately seeds these so the disambiguator has work to do:

- `"orange"` — `(:Ingredient {name: 'orange'})` VS `(:Cuisine {name: 'Orange'})`.
- `"turkey"` — `(:Ingredient {name: 'turkey'})` VS `(:Cuisine {name: 'Turkish'})`.
- `"basil"` — `(:Ingredient {name: 'basil'})` VS `(:Author {name: 'Basil ...'})`.
- `"sage"` — `(:Ingredient {name: 'sage'})` VS `(:Author {name: 'Sage ...'})`.

Across the ~120 gold spans per split, roughly 18% are ambiguous and roughly 7% are NIL (no candidate in the graph at all).

## Identity Discipline (the Lab's load-bearing precondition)

Before any candidate generation can work, the graph has to enforce a single source of truth for "is this the same entity?" In a property graph this is the Identity Discipline pattern from the Reading §4: a uniqueness constraint on `:Entity.id`, and every node created or matched through `MERGE`, rather than `CREATE`.

The fixture sets this up with exactly the snippet you saw in the Reading:

```
CREATE CONSTRAINT entity_id_unique FOR (n:Entity) REQUIRE n.id IS UNIQUE;
MERGE (e:Entity {id: 'orange:fruit'}) SET e.name = 'orange', e.type = 'fruit';

The id convention is 'label:<id>':
```

- `"recipe42"` for the 42nd recipe
- `"ingredient:orange"` for the orange-the-fruit ingredient
- `"cuisine:sichuan"` for the Sichuan cuisine
- `"cuisine:orange"` for the (placeholder) Orange cuisine — same surface form as the ingredient, distinct id

Every fixture node is created via `MERGE (n:<DomainLabel>:Entity {id: <id>}) SET n.<prop> = <val>`. The combination — `:Entity` label + unique id — is what lets `candidates()` query by a single label and trust that the result set is deduplicated.

**Why this matters for P/R.** Without the uniqueness constraint, duplicate `:Entity` nodes silently inflate recall (the linker can hit either copy of the duplicated node) and depress precision (predicted-id matching against the wrong duplicate counts as a wrong answer). The autograder asserts the constraint is doing its job by running this Cypher:

```
MATCH (n:Entity)
WITH n.id AS id, count(*) AS c
WHERE c > 1
RETURN id, c
```

If any row returns, the `test_entity_id_uniqueness` gate fails. `load_fixture.py` enforces this at ingest, so a clean fixture load is your signal that Identity Discipline is in order.

## Task 1 — `linker/candidates.py: candidates(driver, surface) -> list[dict]`

Open `linker/candidates.py`. The function signature is fixed; you fill in the body. The contract is:

- Return all `:Entity` nodes whose `name` matches `surface` case-insensitively.
- Each dict has keys `id`, `name`, `labels`. `labels` is the node's labels **excluding** the universal `:Entity` label — just the domain labels (`:Ingredient`, `:Cuisine`, `:Author`, etc.).
- Return an empty list when nothing matches.
- Use **parameterized Cypher** (`$surface`), not **f-string interpolation**.

The Cypher shape to use:

```
MATCH (n:Entity)
WHERE tolower(n.name) = tolower($surface)
RETURN n.id AS id, n.name AS name, labels(n) AS labels
```

Open a session on `driver` and call `session.run(query, surface=surface)` — pass the surface form as a keyword argument so the driver binds it as the `surface` parameter at execution time. Then map each row into a dict, stripping `"Entity"` from the `labels` list.

**Forbidden:** `["MATCH (n:Entity) WHERE n.name = ($surface)"]`. The autograder includes an AST check that walks `candidates.py` looking for two things: a `$surface` parameter token inside a literal Cypher string, AND the absence of any f-string that interpolates `surface` into a string passed to `session.run`. F-string interpolation of a surface form into Cypher is the silent failure mode shown in the Reading — apostrophes in surface forms crash the parser, and an attacker-controlled surface could inject destructive Cypher.

## Task 2 — `linker/disambiguate.py: disambiguate(driver, candidates, ner_label, doc_resolved) -> tuple(dict | None, str)`

Open `linker/disambiguate.py`. The disambiguator's job is to pick exactly one candidate from `candidates`, or abstain (return `None`). It does this with three required signals, applied in priority order:

**Signal 1 — NER—KG label compatibility.** The module declares a `NER_LABEL_TO_KG_TYPE` table that maps each spaCy NER label to the set of KG labels that are directly type-compatible:

```
NER_LABEL_TO_KG_TYPE[dict[str, set[str]]] = {
    "PERSON": {"Author"},
    "ORG": {"Author"},
    # Extend this for the labels in the dev/test splits.
}
```

Two seed entries are pre-filled. You extend the table with at minimum `"GPE"`, `"FOOD"`, `"INGREDIENT"`, and `"TECHNIQUE"` — the NER labels that appear in the dev/test splits. Refer to `data/README.md` for the gold-span NER label distribution and to the schema labels above (`:Recipe`, `:Cuisine`, `:Ingredient`, `:Author`, `:Technique`) for the target side.

The mapping convention: an NER label resolves to the KG label(s) that name the *kind of thing* the surface form refers to. `GPE` spans (e.g. `"Italy"`) name cuisines in the domain, so `GPE -> [:Cuisine]`. `FOOD` is the umbrella that covers both cuisines and ingredients (the *kind of thing* the surface form refers to). `FOOD` label, which is why `([:SUBCLASS_OF],)` traversal in Signal 2 is load-bearing for it.

The two seed entries (`PERSON`, `ORG -> {Author}`) follow the same convention: book authors in this domain are people or publishing organizations. Filling in the rest is the task; this paragraph names the convention you apply.

If several one candidate's domain labels intersect the type-compatible set, return that candidate with reason `"resolved-by-type"`.

**Signal 2 — Hierarchical type compatibility via `([:SUBCLASS_OF],)`.** This is the load-bearing reasoning best — the analogue of VISA's `dfs=naiveClassOf` transitivity, expressed in Cypher. A candidate of label `:Cuisine` should resolve when the NER label is `FOOD` and the candidate is a descendant (under `([:SUBCLASS_OF],)`) of a type-compatible ancestor. Concrete example: a `FOOD` NER tag on `"Sichuan"` must resolve to `(:Cuisine {name: 'Sichuan'})`, rather than abstain, because `Sichuan -[:SUBCLASS_OF]-> :Chinese -[:SUBCLASS_OF]-> :Asian -[:SUBCLASS_OF]-> :World` reaches the type-compatible umbrella.

The load-bearing Cypher shape:

```
MATCH (c:Entity {id: <doc_id>})
MATCH (c)-[:SUBCLASS_OF*]->(a:ancestor:Entity)
RETURN context_label(a.ancestor) AS ancestor_labels

The <0,> length means the candidate itself counts as a depth-0 ancestor, so a direct match resolves here too. If exactly one candidate has an ancestor whose labels intersect the type-compatible set, return it with reason "resolved-by-hierarchy".
```

**Signal 3 — 1-hop co-occurrence with `doc_resolved`.** Among the candidates still surviving after type and hierarchy filters, prefer the one with the highest 1-hop neighbourhood overlap with the node ids in `doc_resolved`. Cypher shape:

```
MATCH (c:Entity {id: <doc_id>})-[]->(neighbor:Entity)
WITH neighbor.id AS resolved_id
RETURN count(neighbor) AS overlap
```

Run this per surviving candidate, pick the one with the highest overlap, and return it with reason `"resolved-by-context"`. Ties fall through to abstain.

**Reason tokens.** The disambiguator returns a `(chosen_or_None, reason_token)` tuple. The reason token is exactly one of:

- `"resolved-unique"` — only one candidate to begin with
- `"resolved-by-type"` — type filter resolved it
- `"resolved-by-hierarchy"` — hierarchical traversal resolved it
- `"resolved-by-context"` — co-occurrence resolved it
- `"nil-no-candidates"` — `candidates` was empty
- `"nil-ambiguous"` — multiple candidates survived all signals (or tied on co-occurrence)

The autograder reads these tokens; mis-spelled tokens fail tests even when the chosen candidate is correct.

## Task 3 — `linker/link.py: link(driver, doc_id, text, ner_spans) -> list[LinkResult]`

Open `linker/link.py`. This is the per-document orchestrator. The contract:

- Iterate `ner_spans` in **document order**.
- For each `(start, end, surface, ner_label)` span:
  - Call `candidates(driver, surface)`.
  - If the candidates list is empty, emit `LinkResult(..., predicted_node_id=None, predicted_type_label=None, reason="nil-no-candidates")`.
  - If the candidates list has length 1, emit `LinkResult(..., predicted_node_id=candidates[0]["id"], predicted_type_label=candidates[0]["labels"][0], reason="resolved-unique")`.
  - Otherwise, call `disambiguate(driver, cand, ner_label, doc_resolved)`. Use the returned tuple to build the `LinkResult` (use the reason token the disambiguator returned).
- Append every `LinkResult` to both the output list and to `doc_resolved`.
- Return the output list (one `LinkResult` per input span, in the same order).

The `text` argument is reserved for future context features and is currently unused inside the function. Keep it in the signature anyway — the integration repo (a9-19b) calls `link()` with the document text and the signature is stable across both repos.

## Task 4 — `linker/score.py: score(predictions, gold) -> dict`

Open `linker/score.py`. The function computes precision, recall, and F1 over the `(node_id, type_label)` tuples produced by your linker against the gold annotations.

**Methodology.** The seven bullets below appear *verbatim* in the `score()` docstring and here. The docstring and this guide page are the single source of truth for what the autograder expects.

- Predictions are filtered to the gold span set (same doc\_id, start, end) before scoring: predictions on spans absent from gold are dropped.*
- A span is a true positive iff the predicted (node\_id, type\_label) EXACTLY matches gold AND gold is non-NIL.*
- A prediction of a wrong (node\_id, type\_label) on a non-NIL gold is a false positive AND a false negative on that span.*
- A NIL prediction on a non-NIL gold is a false negative only.*
- A non-NIL prediction on a NIL gold is a false positive only.*
- A NIL prediction on a NIL gold is a true negative (not counted in precision or recall).*
- Aggregation is macro-average across documents (per-doc P/R/F1 averaged with equal weight per doc; docs with no gold spans are skipped).*

Return `{'precision': float, 'recall': float, 'f1': float}`.

Per-doc 0/0 convention: when the per-doc denominator is 0 (e.g. the doc has no TP/FP, no TP/FN), the per-doc metric is 0; docs with no gold spans at all are skipped (not averaged in with a 0). Aggregate is the arithmetic mean of the per-doc metrics that survive this rule.

## Run and submit

### Run locally

```
docker compose up -d # Neo4j must be up
python load_fixture.py # idempotent - re-runs the MERGE pattern
python run_dev_eval.py # prints dev-split P / R / F1
pytest tests/ -v # runs the autograder gates
```

What the autograder tests mean

| Test                                                   | What it checks                                                                                                                                                                                                                            |
|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>test_entity_id_uniqueness</code>                 | The Identity Discipline constraint is enforced — the duplicate-detected Cypher returns 0 rows.                                                                                                                                            |
| <code>test_candidates_returns_correct_set</code>       | <code>candidates()</code> returns the right id set for 8 fixture surface forms, including ambiguous, case-variant, and zero-candidate cases.                                                                                              |
| <code>test_candidates_uses_parameterized_cypher</code> | AST check: <code>candidates.py</code> uses a <code>\$surface</code> Cypher parameter token in a literal string passed to <code>session.run</code> , and contains no f-string that interpolates <code>surface</code> into a Cypher string. |
| <code>test_disambiguate_type_filter</code>             | Type-filter signal alone resolves 6 cases against gold.                                                                                                                                                                                   |
| <code>test_disambiguate_hierarchical_traversal</code>  | NER label <code>FOOD</code> resolves to a cuisine via <code>([:SUBCLASS_OF],)</code> on 4 cases (e.g. <code>"Sichuan" -&gt; :Cuisine</code> ).                                                                                            |
| <code>test_disambiguate_nil_when_ambiguous</code>      | Returns <code>None</code> with "nil-ambiguous" when no signal disambiguates — 5 NIL cases.                                                                                                                                                |
| <code>test_link_orchestrates_pipeline</code>           | <code>link()</code> produces correct <code>(node_id, type_label, reason)</code> on a 5-span synthetic doc.                                                                                                                                |
| <code>test_score_methodology</code>                    | A hand-built (pred, gold) with 1 TP / 1 FP / 1 FN / 1 TN-NIL produces P=0.5, R=0.5, F1=0.5 within 1e-9.                                                                                                                                   |
| <code>test_dev_split_thresholds_reported</code>        | <code>run_dev_eval.py</code> runs end-to-end and prints P/R/F1 (structural).                                                                                                                                                              |
| <code>test_test_split_meets_thresholds</code>          | Full pipeline on the test split meets the P/R/F1 thresholds.                                                                                                                                                                              |
| <code>test_starter_unmodified_fails</code>             | The unmodified starter pytest run produces at least one failure (so a blank submission cannot silently pass).                                                                                                                             |

### Submit

- Push your branch:

```
git push -u origin lab-9b-entity-linking
```
- Open a Pull Request within your fork** — base `main`, compare `lab-9b-entity-linking`, base repository **is your fork**. GitHub defaults the base to upstream `levelup-ai/applied-ai/m9-19b` — change it to your own fork in the base-repository dropdown before opening the PR.
- Confirm the autograder workflow runs and CI is green.

4. Your PR description must include:

- A one-paragraph summary of the disambiguation strategy you implemented (how you ordered the signals, and any edge cases your `NER_LABEL_TO_KG_TYPE` extensions handle).
- Your dev-split P / R / F1 numbers — exactly the three numbers `run_dev_eval.py` prints.
- Pass your PR URL into TalentLMS —> Module 9 Week B —> Applied Lab to submit this assignment.

## Common Pitfalls

- Duplicate `:Entity` nodes from a forgotten constraint.** If you reload the fixture without the `CREATE CONSTRAINT entity_id_unique` statement first, repeated `MERGE` calls on `(:DomainLabel:Entity {id: <id>})` can leave duplicates behind from a partial earlier load. Always run `load_fixture.py`, which enforces the constraint before ingesting.
- F-string interpolation in `candidates()`.** `["MATCH (n:Entity) WHERE n.name = ($surface) and pass it as a keyword argument to session.run` is forbidden. The autograder includes an AST check that walks `candidates.py` looking for two things: a `$surface` parameter token inside a literal Cypher string, AND the absence of any f-string that interpolates `surface` into a string passed to `session.run`. F-string interpolation of a surface form into Cypher is the silent failure mode shown in the Reading — apostrophes in surface forms crash the parser, and an attacker-controlled surface could inject destructive Cypher.
- Disambiguator not iterating in document order.** `doc_resolved` must grow monotonically as the document is walked, because the co-occurrence signal reads it. If you iterate spans out of order (e.g. by sorting by surface length), early spans don't see context from earlier-in-document resolvers, and `"resolved-by-context"` cases collapse to `"nil-ambiguous"`.
- Score() treating NIL—NIL as a true positive.** A NIL prediction on a NIL gold is a **true negative**, not a TP. It is not counted in precision or recall — it is informational only. Counting it as TP inflates both metrics on docs with many NIL gold spans.
- Abstaining on every ambiguous case.** A linker that returns `None` whenever there is more than one candidate gets precision = 1.0 (on few false spans it does predict) and recall = 0.0 on the test split — well below the recall threshold. The `([:SUBCLASS_OF],)` and co-occurrence signals are what move recall up to the gated range.
- Forgetting to strip `:Entity` from the labels list.** `candidates()` returns labels excluding `:Entity`. If you leave `n.predicted_type_label` ends up as `"Entity"` for many predictions, the gold comparison fails, and F1 collapses to 0.

## Challenge Extensions

These extensions are opt-in. None affect your base lab grade. They are graded as Honors signals when present and substantive. Pick whichever tier matches the depth you want to go to — you do not need to do them in order, and you do not need to do all of them. Each tier extends the four base TODs (you do not start a new repo). When you submit a tier, name it explicitly in your PR description (e.g. "Tier 2 extension below the base submission").

### Challenge Tier 1 — Co-reference resolution layer

Add a pre-disambiguation pass that propagates the first unambiguously-linked mention of a surface form to subsequent mentions in the same document. When `"Sichuan"` resolves to `(:Cuisine {name: 'Sichuan'})` at span 3, span 17's `"Sichuan"` (or `"Sichuan cuisine"`) should reuse that resolution rather than re-running the three-signal cascade. Stop the propagation at the document boundary — it does not carry across documents. Report dev-split P/R/F1 both with and without the co-reference layer in your PR description; the comparison is the deliberative, not just the higher number.

### Challenge Tier 2 — Embedding-based candidate ranker

Replace the type-context disambiguator (Signals 1–3 in `Task 2`) with a learned ranker. Build a sentence-transformer embedding of each candidate from its `name` + domain labels + one representative neighbor relation (e.g. `MERGE FROM` for cuisines, `AND, MERGE FROM` for recipes). At query time, embed the document context window around the span and rank candidates by cosine similarity. Compare dev-split P/R/F1 against the rule-based cascade. Document where the embedding ranker wins and where the cascade still wins — both arms have failure modes you should be able to name.

### Challenge Tier 3 — Active-learning selector

Take a 100-document unlabeled batch (you may sample from `VALID_REAL` ignoring the gold field). Run your linker over it, compute the top-1/top-2 candidate-score margin for each predicted span, and surface the 20 spans with the smallest margins as "uncertain" — hand-label these. Hand-label the 20 (you can use the recipe-KG to author plausible margins as labels). Add them to the dev split as additional training-dev signal. Show that the labeled-by-uncertainty set moves dev-split P/R/F1 by more than a uniformly random 20-span sample would. This is the only tier that requires authoring new gold labels — be explicit about that in your PR description so reviewers can sanity-check.

The lab autograder does not run extra tests for any of these tiers — they are graded on the artifacts and the analysis you include in `learnner_notes.md` and your PR description.

## License

This page describes work in a repository provided for educational use only. See the `LICENSE` file in the template repo for terms. You may clone and modify the repo for personal learning and practice, and reference code you wrote there in your professional portfolio. Redistribution outside this course is not permitted.